

Eulerian Path Existence

Case study of LLMs use for Math

Samuel Chassot - 26.05.2026

Eulerian Path

Definition

(Undirected and connected) **graph** $G = (V, E)$

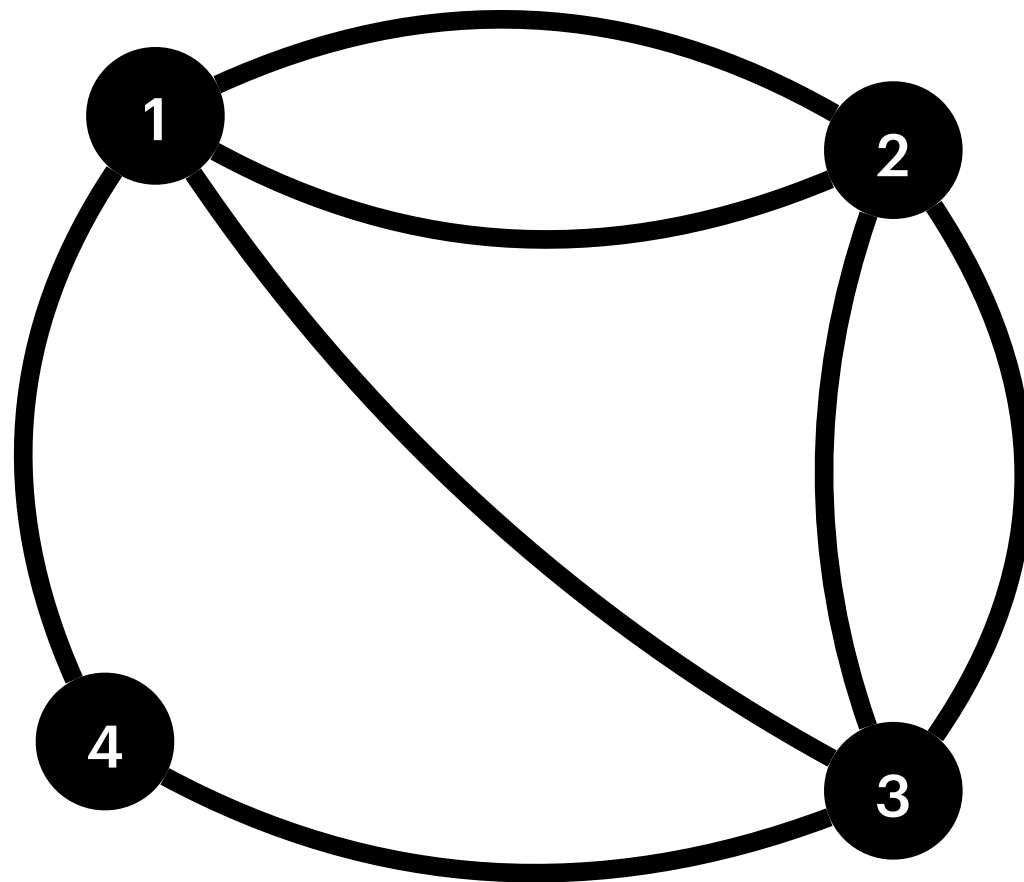
Walk: **sequence of edges** that connects a sequence of vertices

Trail: a **walk** in which **all edges are distinct**

Path: a **trail** that **visits all vertices**

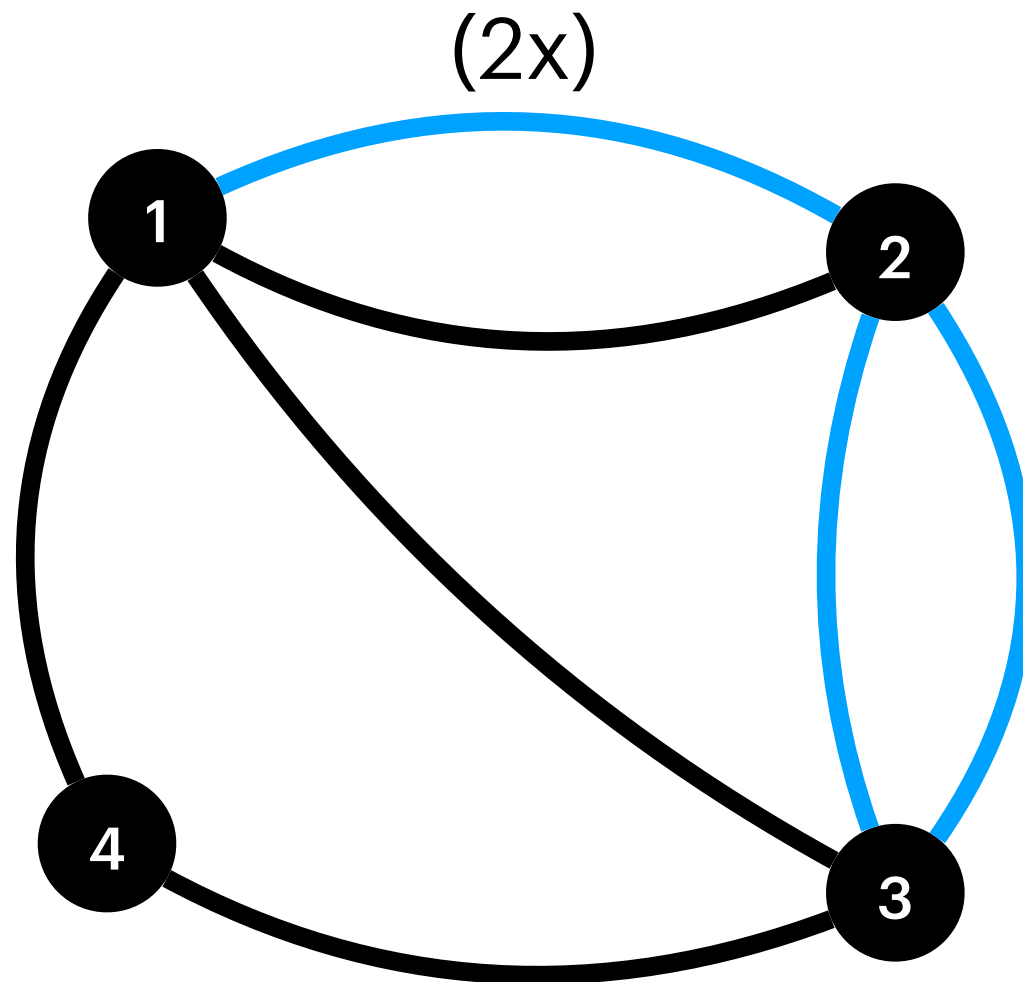
Eulerian Path

Definition



Eulerian Path

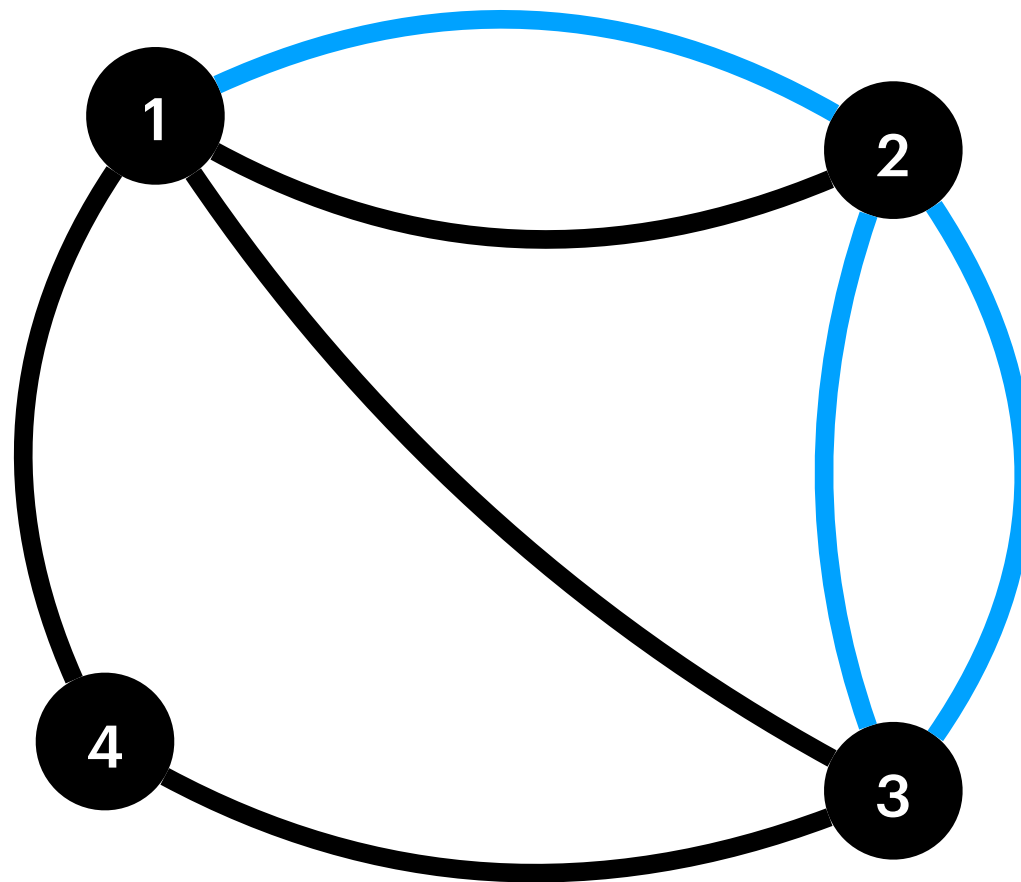
Definition



Walk

Eulerian Path

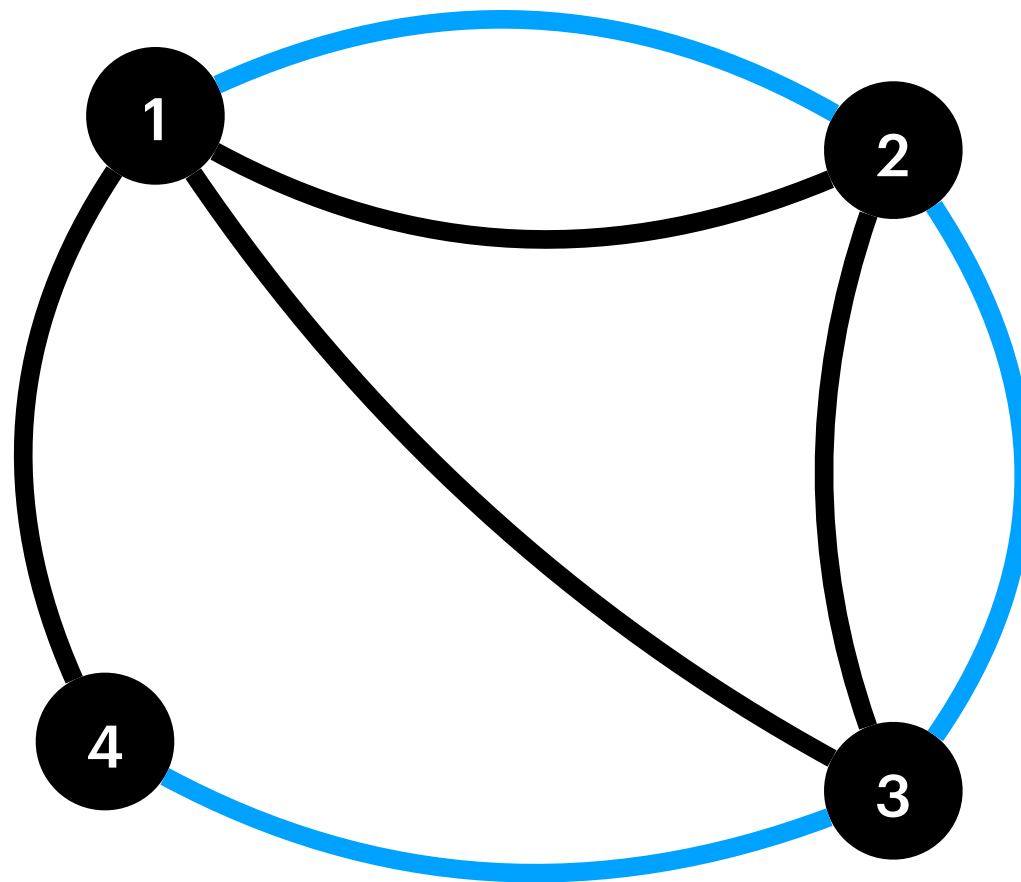
Definition



Trail

Eulerian Path

Definition



Path

Eulerian Path

Definition

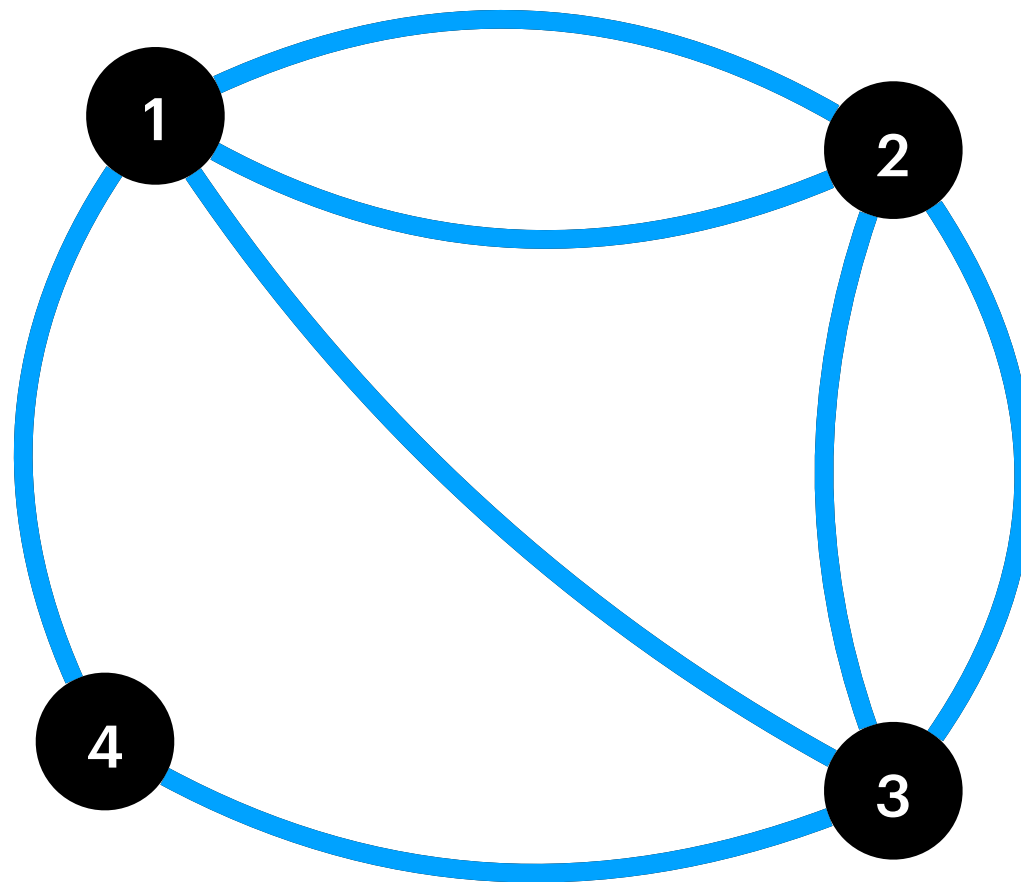
(Undirected and connected) **graph** $G = (V, E)$

Eulerian Path

Trail (and thus a path) that visits all edges **exactly once**

Eulerian Path

Definition



Eulerian Path

Eulerian Path Theorem

Euler's theorem

A connected graph has an Euler path **if and only if all vertices have an even degree**

More general

A connected graph has an Euler path **if and only if the number of vertices with odd degree is 0 or 2**

Proof 1st try

Kimi 2.6

First tried with Kimi 2.6 on EPFL cluster

Only informal theorem definition

From **scratch, without hints**

Proof 1st try

Kimi 2.6

After multiple attempts, it did not reach a proof

It was having trouble with the definitions of graph theory

Proof 2nd attempt

Translation

Next step, I looked for an existing proof

Idea: ask an LLM to **transfer from another proof assistant**

I found out that **this proof was missing from Mathlib**

Proof 2nd attempt

With Mathlib

```
28  ## TODO
```

```
29
```

```
30  * Prove that there exists an Eulerian trail when the conclusion to  
31  | `SimpleGraph.Walk.IsEulerian.card_odd_degree` holds.
```

```
32
```

```
theorem IsEulerian.card_odd_degree [Fintype V] [DecidableRel G.Adj] {u v : V} {p : G.Walk u v}  
  (ht : p.IsEulerian) : Fintype.card { v : V | Odd (G.degree v) } = 0  
  |> Fintype.card { v : V | Odd (G.degree v) } = 2 := by  
  rw [← Set.toFinset_card]  
  apply IsEulerian.card_filter_odd_degree ht  
  simp
```

Proof 2nd attempt

With Mathlib

Did not find an existing formalization

Worked with ChatGPT chat mode to write this proof

Building on top of Mathlib

GPT gave `grep` commands to find **existing definitions**

Proof 2nd attempt

With Mathlib

Resulted in a compiling proof

- First for all even degrees (**~500 loc**)
- Then, for odd = 2, but I had to ask for it (**~1200 loc**)

```
theorem exists_isEulerian_of_connected_card_oddDegree_eq_zero_or_two
  (hconn : G.Connected)
  (hodd : Fintype.card {v : V | Odd (G.degree v)} = 0 ∨
    Fintype.card {v : V | Odd (G.degree v)} = 2) :
  ∃ u v, ∃ p : G.Walk u v, p.IsEulerian
```


Proof 2nd attempt

Translation

Ask GPT to refactor before diving in the proof

- not much shorter but more reused abstractions and less overabstractions

Ask Claude to judge and refactor

- it agrees with the refactoring
- but refactored again $\Rightarrow$ **down to ~650 loc**

Proof sketch

Claude rewrite

```
theorem exists_isEulerian_of_connected_card_odd_degree_eq_zero_or_two
  (hconn : G.Connected)
  (hodd : Fintype.card {v : V | Odd (G.degree v)} = 0 ∨
    Fintype.card {v : V | Odd (G.degree v)} = 2) :
  ∃ u v, ∃ p : G.Walk u v, p.IsEulerian := by
rcases hodd with hzero | htwo
· obtain ⟨u₀⟩ := hconn.nonempty
  have heven := forall_even_degree_of_card_odd_degree_eq_zero (G := G) hzero
  obtain ⟨u, p, hp⟩ :=
    exists_isEulerian_of_connected_forall_even_degree u₀ hconn heven
  exact ⟨u, u, p, hp⟩
· exact exists_isEulerian_of_connected_card_odd_degree_eq_two hconn htwo
```

Proof sketch

All even degree

Take a **maximal trail** (cannot add more edges)

1. it is **closed**.

Contradiction: otherwise end vertex has odd degree

2. it is **Eulerian**

Contradiction: otherwise, add a loop from one vertex
with remaining incoming edges $\rightarrow$ was not maximal

Proof sketch

2 odd degree vertices

Take a **maximal trail p** (cannot add more edges)

1. By parity: start-end vertices are the odd-degree ones

2. it is **Eulerian**

Pick an unused edge connected to p and build a maximal trail with **unused edges**. As all vertices have even degree

→ closed detour, can be added to p (**contradiction**)

Conclusion

Proof of existence of eulerian trail

Self contained problem → nice candidate for this project

Opened a PR to Mathlib

Kimi alone without Mathlib could not do it

ChatGPT with Mathlib gives working proof

Claude refactors to much cleaner proof (and shorter)

Conclusion

Königsberg

